Understanding Search vs. Decision vs. Distinguishing in Post-Quantum Cryptography Hard Problems

1 Why this distinction matters

In post-quantum cryptography (PQC), many security assumptions are stated using one of three flavors of hard problems:

1. **Search problem:** recover the hidden object.
2. **Decision problem:** answer a yes/no question about the instance.
3. **Distinguishing problem:** tell whether samples come from one distribution or another.

These three look similar, but they play different roles in theory and in cryptographic reductions.

Roughly:

$$\text{Search} \quad \Rightarrow \quad \text{Decision} \quad \Rightarrow \quad \text{Distinguishing}$$

in the sense that if you can fully recover the secret, then you can usually answer the weaker yes/no or distinguishing task. The converse is usually harder and may require nontrivial reductions.

2 The core intuition

Suppose Nature hides some secret structure.

- In a **search** problem, you must *find* that hidden structure.
- In a **decision** problem, you only need to *decide whether a certain property holds.*
- In a **distinguishing** problem, you do not need the secret or even a property of one fixed instance; you only need to *tell apart two candidate generating processes.*

So the required output becomes progressively weaker.

3 Formal viewpoint

3.1 Search problems

Definition 1 (Search problem)**.** *A search problem consists of an instance space $\mathcal{I}$ and a relation*

$$R \subseteq \mathcal{I} \times \mathcal{W},$$

where $\mathcal{W}$ is the witness or solution space. Given an instance $I \in \mathcal{I}$, the goal is to output some $w \in \mathcal{W}$ such that

$$(I, w) \in R.$$

The output is not just yes/no. It must be an actual solution.

Example 1 (Search-LWE)**.** *Given samples*

$$(\boldsymbol{a}_i,\, b_i = \langle \boldsymbol{a}_i, \boldsymbol{s} \rangle + e_i \bmod q),$$

recover the secret vector $\boldsymbol{s}$*.*

Example 2 (SIS as search)**.** *Given a matrix* $A \in \mathbb{Z}_q^{m \times n}$*, find a nonzero short vector* $\boldsymbol{z} \in \mathbb{Z}^n$ *such that*

$$A\boldsymbol{z} \equiv \boldsymbol{0} \pmod q.$$

3.2 Decision problems

Definition 2 (Decision problem)**.** *A decision problem is specified by a language* $L \subseteq \mathcal{I}$*. Given an instance* I*, the goal is to output*

$$\begin{cases} 1 & \text{if } I \in L, \\ 0 & \text{if } I \notin L. \end{cases}$$

The output is only one bit.

Example 3 (GapSVP)**.** *Given a lattice basis* B *and threshold* d*, decide whether the shortest nonzero lattice vector is*

$$\lambda_1(\mathcal{L}(B)) \le d$$

or

$$\lambda_1(\mathcal{L}(B)) > \gamma d,$$

for some approximation factor $\gamma > 1$*, assuming the input is promised to satisfy one of these two cases.*

Remark 1. *This is not a search problem because you do* not *need to output the short vector itself. You only need to decide which case holds.*

3.3 Distinguishing problems

Definition 3 (Distinguishing problem)**.** *A distinguishing problem gives oracle access or samples from one of two distributions*

$$\mathcal{D}_0, \mathcal{D}_1.$$

The goal is to decide from which distribution the samples were drawn, with non-negligible advantage over random guessing.

This is often written as:

$$|\Pr[\mathcal{A}(\mathcal{D}_0) = 1] - \Pr[\mathcal{A}(\mathcal{D}_1) = 1]|$$

being non-negligible.

Example 4 (Decision-LWE as a distinguishing problem)**.** *Given samples, distinguish between*

$$(\boldsymbol{a},\, \langle \boldsymbol{a}, \boldsymbol{s} \rangle + e \bmod q)$$

and

$$(\boldsymbol{a},\, u),$$

where $u \leftarrow \mathbb{Z}_q$ *is uniform.*

Remark 2. *In modern cryptography, many so-called* ***decision assumptions*** *are really* ***distribution distinguishing assumptions****. Decision-LWE is the canonical example.*

4 A clean conceptual separation

4.1 Search asks "What is the hidden object?"

The hidden object may be:

- a secret vector $\boldsymbol{s}$ in LWE,
- a short relation $\boldsymbol{z}$ in SIS,
- a short lattice vector in SVP,
- a close lattice vector in CVP.

The algorithm must actually produce it.

4.2 Decision asks "Does this instance have the property?"

Examples:

- Is there a vector shorter than threshold d?
- Is this lattice instance in the YES-case or NO-case?
- Is this polynomial system solvable under some promise?

4.3 Distinguishing asks "Which world am I in?"

You are shown samples from one of two worlds:

- **structured world** (contains hidden algebraic or noisy linear structure),
- **random world** (uniform or pseudorandom noise only).

The task is to identify the world, not necessarily to recover the structure.

5 The standard cryptographic examples

5.1 LWE

Let $\boldsymbol{s} \in \mathbb{Z}_q^n$ be secret. Samples are of the form

$$(\boldsymbol{a}, b), \qquad b = \langle \boldsymbol{a}, \boldsymbol{s} \rangle + e \pmod{q},$$

where $\boldsymbol{a} \leftarrow \mathbb{Z}_q^n$ uniformly and e is drawn from some error distribution.

Search-LWE. Given many samples $(\boldsymbol{a}_i, b_i)$, recover $\boldsymbol{s}$.

Decision-LWE / Distinguishing-LWE. Given many samples, decide whether they are:

- valid LWE samples with a fixed secret $\boldsymbol{s}$, or
- uniformly random samples in $\mathbb{Z}_q^n \times \mathbb{Z}_q$.

Interpretation.

- Search-LWE asks for the *secret*.
- Decision-LWE asks whether the sample set is *structured* or *random*.

5.2 SIS

Let $A \in \mathbb{Z}_q^{m \times n}$.

Search-SIS. Find a short nonzero vector $\boldsymbol{z} \in \mathbb{Z}^n$ such that

$$A\boldsymbol{z} \equiv \mathbf{0} \pmod{q}.$$

Decision analogue? SIS is usually used primarily as a *search* assumption. A natural decision-style version would be something like:

> Does there exist a nonzero vector $\boldsymbol{z}$ with $\|\boldsymbol{z}\| \leq \beta$ such that $A\boldsymbol{z} \equiv \mathbf{0} \pmod{q}$?

But in cryptography, SIS is most often stated and used in its search form.

5.3 Lattice geometry problems

SVP (Shortest Vector Problem). Given a lattice basis B, output a shortest nonzero vector in $\mathcal{L}(B)$.

This is a **search** problem.

GapSVP. Given B and threshold d, decide whether

$$\lambda_1(\mathcal{L}(B)) \leq d$$

or

$$\lambda_1(\mathcal{L}(B)) > \gamma d.$$

This is a **decision** problem with a promise gap.

CVP (Closest Vector Problem). Given basis B and target $\boldsymbol{t}$, output a lattice vector closest to $\boldsymbol{t}$.

This is a **search** problem.

BDD (Bounded Distance Decoding). Given a target known to be unusually close to the lattice, recover the nearby lattice point.

This is also fundamentally a **search** problem.

6 Why distinguishing is not exactly the same as ordinary decision

Students often say:

> "A distinguishing problem is just a decision problem, because the answer is yes/no."

This is *partly true* but conceptually incomplete.

6.1 Common ground

Both decision and distinguishing output one bit. So at the most abstract complexity-theoretic level, a distinguisher computes a Boolean function.

6.2 Important difference

In a **decision problem**, the input is usually a *single instance* and you ask whether it belongs to a language.

In a **distinguishing problem**, the input is typically generated from one of *two probability distributions*. The challenge is statistical or computational separation of distributions.

So:

- **Decision** is language-based.
- **Distinguishing** is distribution-based.

In cryptography, the distribution-based viewpoint is often the more natural one.

7 Reduction intuition between the three

7.1 From search to distinguishing

If you can solve the search problem, you can often distinguish.

Example 5 (Search-LWE $\Rightarrow$ Distinguishing-LWE)**.** *Suppose you can recover $\boldsymbol{s}$. Then, on input samples $(\boldsymbol{a}_i, b_i)$:*

1. *Recover candidate $\boldsymbol{s}$.*
2. *Check whether each*
$$b_i - \langle \boldsymbol{a}_i, \boldsymbol{s} \rangle \pmod{q}$$
looks like small noise.

If yes, declare "LWE"; otherwise declare "uniform".

So search is at least as strong as distinguishing in this sense.

7.2 From distinguishing to search

This direction is much subtler. One usually needs a theorem stating that a distinguisher can be transformed into a search solver.

For LWE, there are standard reductions showing that under appropriate parameters, *decision-LWE and search-LWE are polynomial-time equivalent.*

Remark 3. *This equivalence is nontrivial. It does not follow merely from intuition; it requires a formal reduction.*

7.3 From search to ordinary decision

If you can find the exact witness, then you can certainly answer whether one exists or whether the relevant threshold is met.

For example:

$$\text{SVP solver} \Rightarrow \text{GapSVP solver}.$$

But the reverse need not hold in any obvious way, because knowing only that a short vector exists does not directly tell you what the vector is.

8 A geometric picture

8.1 Search in lattices

You are standing outside a lattice and must point to a specific geometric object:

- the shortest nonzero lattice vector,
- the lattice point closest to a target,
- a short modular dependency.

8.2 Decision in lattices

You do not need the vector. You only answer:

> "Is there a vector this short?"

8.3 Distinguishing in noisy linear algebra

You are shown clouds of samples:

- one cloud comes from a hidden affine/noisy structure,
- the other cloud is uniform noise.

You only decide which cloud-generating mechanism is more likely.

9 Practical role in cryptography

9.1 Search assumptions

Search assumptions are common when the cryptographic break itself requires producing a concrete object.

Examples:

- forging a signature may amount to finding a short SIS solution,
- decoding may amount to finding a close vector,
- key recovery is a search task.

9.2 Distinguishing assumptions

Distinguishing assumptions are common in encryption and pseudorandomness.

Examples:

- IND-CPA security is itself a distinguishing game,
- LWE naturally gives pseudorandom-looking samples,
- module-LWE and ring-LWE assumptions are usually phrased as distinguishing assumptions.

9.3 Decision assumptions

Decision versions often appear in complexity theory and worst-case to average-case reductions.
Examples:

- GapSVP,
- GapCVP,
- approximate threshold formulations used in hardness proofs.

10 Summary table

Type	Output	Typical question	PQC examples
Search	explicit witness / secret / vector	"Find the hidden object"	Search-LWE, SIS, SVP, CVP, BDD
Decision	one bit about instance property	"Does this property hold?"	GapSVP, GapCVP, threshold versions of lattice problems
Distinguishing	one bit about source distribution	"Are these samples structured or random?"	Decision-LWE, Ring-LWE, Module-LWE

11 A very important warning

In PQC papers, the word **decision** is sometimes used loosely.

- In classical complexity theory, "decision" usually means language membership.
- In cryptography, "decision-LWE" usually means distinguishing LWE samples from uniform.

So when reading a paper, always ask:

Is "decision" here an *instance property problem*, or a *distribution distinguishing problem*?

12 One-line memory aid

Search :	recover the object,
Decision :	decide the property,
Distinguishing :	identify the distribution.

13 Mini-exercises

1. Explain why recovering the LWE secret immediately gives a distinguisher.
2. Explain why GapSVP is a decision problem but SVP is a search problem.
3. For SIS, formulate a natural yes/no version and compare it with the usual search version.
4. In one sentence each, state the difference between:
   - Search-LWE and Decision-LWE,
   - SVP and GapSVP,
   - decision and distinguishing.

14 Final takeaway

The cleanest way to think about PQC hard problems is:

- **Search:** produce the hidden mathematical object.
- **Decision:** answer a yes/no structural question about one instance.
- **Distinguishing:** tell whether observed data comes from a structured source or a random source.

For lattice-based cryptography in particular:

- **SVP/CVP/SIS** are naturally search-oriented,
- **GapSVP/GapCVP** are naturally decision-oriented,
- **LWE/RLWE/MLWE** are most often used in distinguishing form.

15 Code-based hard problems: search vs. decision vs. distinguishing

Code-based cryptography gives another clean setting in which the distinction between search, decision, and distinguishing becomes very important.

15.1 Basic coding-theory setup

Let $\mathbb{F}_q$ be a finite field, often $\mathbb{F}_2$ in classical code-based cryptography.

A linear code C of length n and dimension k is a k-dimensional subspace

$$C \subseteq \mathbb{F}_q^n.$$

It is commonly represented by either:

- a **generator matrix** $G \in \mathbb{F}_q^{k\times n}$ such that
$$C = \{\boldsymbol{m}G : \boldsymbol{m} \in \mathbb{F}_q^k\},$$
- or a **parity-check matrix** $H \in \mathbb{F}_q^{(n-k)\times n}$ such that
$$C = \{\boldsymbol{c} \in \mathbb{F}_q^n : H\boldsymbol{c}^T = \boldsymbol{0}\}.$$

The *Hamming weight* of a vector $\boldsymbol{e} \in \mathbb{F}_q^n$, written $w_H(\boldsymbol{e})$, is the number of nonzero coordinates.

In code-based cryptography, the hidden object is often a **low-weight error vector**.

15.2 The central problem: syndrome decoding

Given a parity-check matrix H and a syndrome $\boldsymbol{s}$, the fundamental equation is

$$H\boldsymbol{e}^T = \boldsymbol{s}^T.$$

The unknown is the error vector $\boldsymbol{e}$.

Why this matters. If $\boldsymbol{y} = \boldsymbol{c} + \boldsymbol{e}$ is a noisy codeword, then

$$H\boldsymbol{y}^T = H(\boldsymbol{c}+\boldsymbol{e})^T = H\boldsymbol{c}^T + H\boldsymbol{e}^T = \boldsymbol{0} + H\boldsymbol{e}^T = \boldsymbol{s}^T.$$

So the syndrome depends only on the error pattern, not on the codeword itself.

15.3 Search version: Syndrome Decoding (SD)

Definition 4 (Search Syndrome Decoding)**.** *Given*

$$H \in \mathbb{F}_q^{(n-k)\times n}, \qquad \boldsymbol{s} \in \mathbb{F}_q^{n-k}, \qquad w,$$

find a vector $\boldsymbol{e} \in \mathbb{F}_q^n$ *such that*

$$H\boldsymbol{e}^T = \boldsymbol{s}^T \qquad \text{and} \qquad w_H(\boldsymbol{e}) \le w.$$

This is a **search problem**: you must output the actual low-weight error vector.

Remark 4. *This is the code-based analogue of many PQC search problems:*

- *in LWE search, recover the secret;*
- *in SIS, find a short relation;*
- *in syndrome decoding, find the sparse error.*

15.4 Decision version

A natural decision analogue is:

Definition 5 (Decision Syndrome Decoding)**.** *Given*

$$H, \boldsymbol{s}, w,$$

decide whether there exists some vector $\boldsymbol{e}$ *such that*

$$H\boldsymbol{e}^T = \boldsymbol{s}^T \qquad \text{and} \qquad w_H(\boldsymbol{e}) \leq w.$$

Here the output is only:

YES or NO.

You do not need to produce the error vector.

15.5 Gap-style decision version

Just as lattices have GapSVP and GapCVP, code problems can also be phrased in threshold or promise-gap form.

Definition 6 (Gap decoding style problem)**.** *Given* $H, \boldsymbol{s}, w$ *and approximation factor* $\gamma > 1$*, distinguish between:*

- ***YES-case:*** *there exists* $\boldsymbol{e}$ *with*

$$H\boldsymbol{e}^T = \boldsymbol{s}^T \quad \text{and} \quad w_H(\boldsymbol{e}) \leq w,$$

- ***NO-case:*** *every* $\boldsymbol{e}$ *satisfying*

$$H\boldsymbol{e}^T = \boldsymbol{s}^T$$

has

$$w_H(\boldsymbol{e}) > \gamma w.$$

This is conceptually parallel to lattice gap problems.

15.6 Distinguishing version

In code-based cryptography, distinguishing problems often ask whether some public object comes from a *structured code family* or from a *random linear code.*

Generic form. Given some matrix or samples, distinguish between:

- **structured world:** generated from a code with hidden algebraic structure,
- **random world:** generated uniformly or from an unstructured random code ensemble.

15.7 Example: McEliece-style viewpoint

In the McEliece framework, one starts with a secret code that has efficient decoding structure, then disguises it publicly.

Very roughly:

- the **secret key** contains a highly structured code with an efficient decoder,
- the **public key** looks like a generator matrix of a random linear code.

A natural distinguishing question is:

Can an adversary distinguish the public code from a uniformly random linear code?

That is a **distinguishing problem**, not a search problem.

Remark 5. *This is different from syndrome decoding:*

- ***Syndrome decoding*** *asks to recover a low-weight error;*
- ***code indistinguishability*** *asks whether the code itself hides special algebraic structure.*

15.8 Geometric intuition in Hamming space

Lattice problems live in Euclidean geometry. Code-based problems live in **Hamming geometry**.

- A sparse error vector means a vector with few nonzero coordinates.
- Decoding asks for a codeword close to the received word in Hamming distance.
- Syndrome decoding asks for the sparse error pattern that explains the syndrome.

So the analogies are:

$$\text{short vector in lattices} \longleftrightarrow \text{low-weight vector in codes},$$

$$\text{Euclidean distance} \longleftrightarrow \text{Hamming distance},$$

$$\text{closest vector problem} \longleftrightarrow \text{nearest codeword / decoding problem}.$$

15.9 Search vs. decision vs. distinguishing in code-based PQC

Search. Recover the hidden sparse object.
Typical question:

Find $\boldsymbol{e}$ such that $H\boldsymbol{e}^T = \boldsymbol{s}^T$ and $w_H(\boldsymbol{e}) \leq w$.

Decision. Determine whether such a sparse object exists.
Typical question:

Does there exist an error vector of weight at most w consistent with this syndrome?

Distinguishing. Tell structured-code distributions apart from random-code distributions.
Typical question:

Does this public matrix come from a hidden Goppa-like construction, or is it just a random linear code?

15.10 Why the distinction matters in cryptography

These three forms appear in different security roles.

- **Search hardness** is central when breaking the scheme means recovering an error pattern or decoding.
- **Decision hardness** is useful for complexity-theoretic classification and threshold formulations.
- **Distinguishing hardness** is relevant when public keys are supposed to look random and leak no visible structure.

15.11 Comparison with lattice-based assumptions

The analogy with lattice-based cryptography is very useful:

Category	**Lattice-based**	**Code-based**
Search	SVP, CVP, SIS, Search-LWE	Syndrome Decoding, low-weight codeword search, decoding
Decision	GapSVP, GapCVP	existence of error of weight $\leq w$, gap decoding formulations
Distinguishing	Decision-LWE, Ring-LWE, Module-LWE	structured code vs. random code, public-key indistinguishability

15.12 A very important caution

In code-based cryptography, not every assumption is naturally a distinguishing assumption. For example:

- **syndrome decoding** is fundamentally a *search* problem;
- **McEliece key indistinguishability** is fundamentally a *distinguishing* problem.

So one should not compress all code-based hardness into a single category.

15.13 One-line memory aid for code-based PQC

Search :	find the low-weight error,
Decision :	decide whether such an error exists,
Distinguishing :	tell structured codes from random codes.

15.14 Mini-exercises on code-based problems

1. Explain why recovering the error vector in syndrome decoding immediately solves the corresponding decision problem.
2. State the difference between:
   - decoding a received word,
   - syndrome decoding,
   - distinguishing a structured code from a random code.
3. Give the Hamming-space analogue of:
   - a short lattice vector,
   - a closest lattice vector,
   - a gap decision problem.

16 Summary table across PQC families

Type	**Output**	**Lattice-based examples**	**Code-based examples**
Search	explicit witness / secret / sparse object	SVP, CVP, BDD, SIS, Search-LWE	Syndrome Decoding, decoding, low-weight codeword search
Decision	one bit about instance property	GapSVP, GapCVP	existence of error of weight $\leq w$, gap decoding forms
Distinguishing	one bit about source distribution	Decision-LWE, Ring-LWE, Module-LWE	structured code vs. random code, disguised secret code vs. random linear code

17 Final takeaway

The cleanest way to think about PQC hard problems is:

- **Search:** produce the hidden mathematical object.
- **Decision:** answer a yes/no structural question about one instance.
- **Distinguishing:** tell whether observed data comes from a structured source or a random source.

Across major PQC families:

- **lattice-based:** SVP/CVP/SIS are search-oriented, GapSVP is decision-oriented, and LWE-type assumptions are often distinguishing-oriented;
- **code-based:** syndrome decoding is search-oriented, threshold decoding variants are decision-oriented, and code indistinguishability questions are distinguishing-oriented.

18 Compact summary table

19 PQC hard problems classified as Search, Decision, and Distinguishing

PQC family	Representative hard problem / assumption	Type	Informal task	Typical formulation
Lattice-based	SVP (Shortest Vector Problem)	Search	Find the shortest nonzero lattice vector	Given a basis B of a lattice output $\boldsymbol{v} \in \mathcal{L}(B) \setminus \{\mathbf{0}\}$ such $\|\boldsymbol{v}\| = \lambda_1(\mathcal{L}(B))$.
Lattice-based	Approx-SVP	Search	Find a sufficiently short nonzero lattice vector	Given B, output $\boldsymbol{v} \neq \mathbf{0}$ with $\gamma\lambda_1(\mathcal{L}(B))$ for approximation $\gamma \geq 1$.
Lattice-based	GapSVP	Decision	Decide whether the lattice has a very short vector	Given (B, d), distinguish be $\lambda_1(\mathcal{L}(B)) \leq d$ and $\lambda_1(\mathcal{L}(B))$ under a promise.
Lattice-based	CVP (Closest Vector Problem)	Search	Find the lattice point closest to a target	Given a basis B and target $\boldsymbol{t}$, $\boldsymbol{v} \in \mathcal{L}(B)$ minimizing $\|\boldsymbol{t} - \boldsymbol{v}\|$.
Lattice-based	Approx-CVP	Search	Find a lattice vector sufficiently close to the target	Given $(B, \boldsymbol{t})$, output $\boldsymbol{v} \in \mathcal{L}(B$ $\|\boldsymbol{t} - \boldsymbol{v}\| \leq \gamma \cdot \mathrm{dist}(\boldsymbol{t}, \mathcal{L}(B))$.
Lattice-based	GapCVP	Decision	Decide whether the target is very close or definitely far from the lattice	Given $(B, \boldsymbol{t}, d)$, distinguish be $\mathrm{dist}(\boldsymbol{t}, \mathcal{L}(B)) \leq d$ and $> \gamma d$, a promise.
Lattice-based	BDD (Bounded Distance Decoding)	Search	Recover the unique nearby lattice point	Given $(B, \boldsymbol{t})$ with promise tha unusually close to some uniqu $\mathcal{L}(B)$, output that $\boldsymbol{v}$.
Lattice-based	SIS (Short Integer Solution)	Search	Find a short nonzero modular relation	Given $A \in \mathbb{Z}_q^{m \times n}$, find $\boldsymbol{z} \in \mathbb{Z}^n$ such that $A\boldsymbol{z} \equiv \mathbf{0} \pmod{q}$ an is small.
Lattice-based	Decision-SIS style variant	Decision	Decide whether a short modular dependency exists	Given (A, β), decide whether exists nonzero $\boldsymbol{z}$ with $A\boldsymbol{z}$ $\pmod{q}$ and $\|\boldsymbol{z}\| \leq \beta$.
Lattice-based	Search-LWE	Search	Recover the hidden secret from noisy linear equations	Given samples $(\boldsymbol{a}_i, b_i)$ where $\langle \boldsymbol{a}_i, \boldsymbol{s} \rangle + e_i \pmod{q}$, recover $\boldsymbol{s}$
Lattice-based	Decision-LWE	Distinguishing	Tell LWE samples from uniform random samples	Distinguish $(\boldsymbol{a}, \langle \boldsymbol{a}, \boldsymbol{s} \rangle + e \bmod q$ $(\boldsymbol{a}, u)$ where $u \leftarrow \mathbb{Z}_q$ uniformly
Lattice-based	Search-RLWE / Search-MLWE	Search	Recover the secret ring/module element	Given noisy ring or module samples, recover the hidden sec ement/vector.
Lattice-based	Decision-RLWE / Decision-MLWE	Distinguishing	Distinguish structured noisy algebraic samples from random ones	Distinguish valid RLWE/M samples from uniformly ra samples over the same space.
Code-based	Syndrome Decoding (SD)	Search	Find a low-weight error matching the syndrome	Given $H, \boldsymbol{s}, w$, find $\boldsymbol{e}$ such $H\boldsymbol{e}^T = \boldsymbol{s}^T$ and $w_H(\boldsymbol{e}) \leq w$.
Code-based	Decoding / Nearest Codeword Problem	Search	Find the codeword closest to a received word	Given a code C and received w output $\boldsymbol{c} \in C$ minimizing Ham distance $d_H(\boldsymbol{y}, \boldsymbol{c})$.
Code-based	Low-Weight Codeword Problem	Search	Find a nonzero codeword of small Hamming weight	Given a linear code C, nonzero $\boldsymbol{c} \in C$ with small $w_H($
Code-based	Decision-SD	Decision	Decide whether a low-weight error exists	Given $(H, \boldsymbol{s}, w)$, decide whether exists $\boldsymbol{e}$ such that $H\boldsymbol{e}^T = \boldsymbol{s}$ $w_H(\boldsymbol{e}) \leq w$.
Code-based	Gap decoding variant	Decision	Decide whether decoding radius is small or definitely larger	Given an instance and thresh distinguish between existence o lution of weight $\leq w$ and no tence of any solution of weight
Code-based	Structured-code indistinguishability	Distinguishing	Tell whether a public code hides algebraic structure or is random	Distinguish a disguised stru code family from a uniformly ra linear code.
Code-based	McEliece public-key indistinguishability viewpoint	Distinguishing	Distinguish a public generator/parity-check matrix from a random one	Given a public matrix, whether it comes from a efficiently decodable structure or from a random code ensemb
Multivariate	MQ (Multivariate Quadratic solving)	Search	Find a solution to a system of multivariate quadratic equations	Given quadratic polyn $f_1, \ldots, f_m$ over $\mathbb{F}_q$, find $\boldsymbol{x}$ that $f_i(\boldsymbol{x}) = 0$ for all i.
Multivariate	Decision-MQ	Decision	Decide whether the polynomial system has a solution	Given $f_1, \ldots, f_m$, decide w there exists $\boldsymbol{x}$ such that $f_i(\boldsymbol{x}$ for every i.
Multivariate	Oil/Vinegar structure distinguishability style question	Distinguishing	Tell whether a public polynomial map has hidden structure or is random-looking	Distinguish a public key gen from a structured trapdoor struction from a uniformly ra quadratic system.
Isogeny-based	Path-finding / hidden isogeny recovery	Search	Recover the secret isogeny or path in the isogeny graph	Given public elliptic-curve da lated by an unknown isogeny, pute the connecting isogeny or

Family	Problem	Type	What the adversary must do
Lattice-based	SVP, Approx-SVP, CVP, BDD, SIS, Search-LWE	Search	Recover a shortest/close vector, a short modular relation, or an LWE secret.
Lattice-based	GapSVP, GapCVP, decision-style SIS variants	Decision	Answer whether a short vector / close vector / short relation exists below a threshold.
Lattice-based	Decision-LWE, Decision-RLWE, Decision-MLWE	Distinguishing	Tell structured noisy samples from uniformly random samples.
Code-based	Syndrome Decoding, nearest codeword decoding, low-weight codeword search	Search	Recover a sparse error, decode a received word, or find a low-weight codeword.
Code-based	Decision-SD, gap decoding variants	Decision	Decide whether a low-weight error or nearby codeword exists.
Code-based	Structured code vs. random code, McEliece public-key indistinguishability	Distinguishing	Tell hidden algebraic code structure from a random linear code.
Multivariate	MQ solving	Search	Find a solution to a system of multivariate quadratic equations.
Multivariate	Decision-MQ	Decision	Decide whether the quadratic system has any solution.
Multivariate	Structured public map vs. random quadratic map	Distinguishing	Tell whether the public polynomial system hides trapdoor structure.
Isogeny-based	Secret isogeny / path recovery	Search	Recover the hidden isogeny or secret path in the graph.
Isogeny-based	Supersingularity-type membership questions	Decision	Decide whether a given object has a target structural property.
Isogeny-based	Structured public data vs. random object distributions	Distinguishing	Tell cryptosystem-generated objects from random ambient objects.
Hash-based	Preimage, second-preimage, collision finding	Search	Find an input or pair of inputs satisfying the required hash relation.
Hash-based	Promise-style existence questions	Decision	Decide whether a valid witness exists under the given promise.
Hash-based	Random-oracle / pseudorandom-style distinguishability views	Distinguishing	Tell whether outputs come from a specific construction or an ideal random source.

Reading guide. A *search* problem asks the adversary to recover an explicit hidden object. A *decision* problem asks only for a yes/no answer about whether an instance has a target property. A *distinguishing* problem asks the adversary to tell apart two distributions, usually a structured distribution and a random one.

20 PQC hard problems classified as Search, Decision, and Distinguishing

PQC family	Representative hard problem / assumption	Type	Informal task	Typical formulation
Lattice-based	SVP (Shortest Vector Problem)	Search	Find the shortest nonzero lattice vector	Given a basis B of a lattice $\mathcal{L}(B)$, output $\boldsymbol{v} \in \mathcal{L}(B) \setminus \{\mathbf{0}\}$ such that $\|\boldsymbol{v}\| = \lambda_1(\mathcal{L}(B))$.
	Approx-SVP	Search	Find a sufficiently short nonzero lattice vector	Given B, output $\boldsymbol{v} \neq \mathbf{0}$ with $\|\boldsymbol{v}\| \leq \gamma\lambda_1(\mathcal{L}(B))$.
	GapSVP	Decision	Decide whether the lattice contains a very short vector	Given (B, d), distinguish between $\lambda_1(\mathcal{L}(B)) \leq d$ and $\lambda_1(\mathcal{L}(B)) > \gamma d$, under a promise.
	CVP (Closest Vector Problem)	Search	Find the lattice vector closest to a target	Given a basis B and target $\boldsymbol{t}$, output $\boldsymbol{v} \in \mathcal{L}(B)$ minimizing $\|\boldsymbol{t} - \boldsymbol{v}\|$.
	Approx-CVP	Search	Find a lattice vector sufficiently close to the target	Given $(B, \boldsymbol{t})$, output $\boldsymbol{v} \in \mathcal{L}(B)$ with $\|\boldsymbol{t} - \boldsymbol{v}\| \leq \gamma \cdot \mathrm{dist}(\boldsymbol{t}, \mathcal{L}(B))$.
	GapCVP	Decision	Decide whether the target is very close or definitely far	Given $(B, \boldsymbol{t}, d)$, distinguish between $\mathrm{dist}(\boldsymbol{t}, \mathcal{L}(B)) \leq d$ and $> \gamma d$, under a promise.
	BDD (Bounded Distance Decoding)	Search	Recover the unique nearby lattice point	Given $(B, \boldsymbol{t})$ with a uniqueness promise, output the lattice vector closest to $\boldsymbol{t}$.
	SIS (Short Integer Solution)	Search	Find a short nonzero modular relation	Given $A \in \mathbb{Z}_q^{m \times n}$, find $\boldsymbol{z} \in \mathbb{Z}^n \setminus \{\mathbf{0}\}$ such that $A\boldsymbol{z} \equiv \mathbf{0} \pmod{q}$ and $\|\boldsymbol{z}\|$ is small.
	Decision-SIS style variant	Decision	Decide whether a short modular dependency exists	Given (A, β), decide whether there exists nonzero $\boldsymbol{z}$ with $A\boldsymbol{z} \equiv \mathbf{0} \pmod{q}$ and $\|\boldsymbol{z}\| \leq \beta$.
	Search-LWE	Search	Recover the hidden secret from noisy linear equations	Given samples $(\boldsymbol{a}_i, b_i)$ where $b_i = \langle \boldsymbol{a}_i, \boldsymbol{s} \rangle + e_i \pmod{q}$, recover $\boldsymbol{s}$.
	Decision-LWE	Distinguishing	Tell LWE samples from uniform random samples	Distinguish $(\boldsymbol{a}, \langle \boldsymbol{a}, \boldsymbol{s} \rangle + e \bmod q)$ from $(\boldsymbol{a}, u)$ where $u \leftarrow \mathbb{Z}_q$ uniformly.
	Decision-RLWE / Decision-MLWE	Distinguishing	Distinguish structured noisy algebraic samples from random ones	Distinguish valid RLWE/MLWE samples from uniformly random samples over the same space.
Code-based	Syndrome Decoding (SD)	Search	Find a low-weight error matching the syndrome	Given $H, \boldsymbol{s}, w$, find $\boldsymbol{e}$ such that $H\boldsymbol{e}^T = \boldsymbol{s}^T$ and $w_H(\boldsymbol{e}) \leq w$.

21 PQC hard problems classified as Search, Decision, and Distinguishing

PQC family	Type	Representative hard problem / assumption	Informal task	Typical formulation
Lattice-based	Search	SVP (Shortest Vector Problem)	Find the shortest nonzero lattice vector	Given a basis B of a lattice $\mathcal{L}(B)$, output $\boldsymbol{v} \in \mathcal{L}(B) \setminus \{\mathbf{0}\}$ such that $\|\boldsymbol{v}\| = \lambda_1(\mathcal{L}(B))$.
		Approx-SVP	Find a sufficiently short nonzero lattice vector	Given B, output $\boldsymbol{v} \neq \mathbf{0}$ with $\|\boldsymbol{v}\| \leq \gamma\lambda_1(\mathcal{L}(B))$.
		CVP (Closest Vector Problem)	Find the lattice vector closest to a target	Given a basis B and target $\boldsymbol{t}$, output $\boldsymbol{v} \in \mathcal{L}(B)$ minimizing $\|\boldsymbol{t} - \boldsymbol{v}\|$.
		Approx-CVP	Find a lattice vector sufficiently close to the target	Given $(B, \boldsymbol{t})$, output $\boldsymbol{v} \in \mathcal{L}(B)$ with $\|\boldsymbol{t} - \boldsymbol{v}\| \leq \gamma \cdot \mathrm{dist}(\boldsymbol{t}, \mathcal{L}(B))$.
		BDD (Bounded Distance Decoding)	Recover the unique nearby lattice point	Given $(B, \boldsymbol{t})$ with a uniqueness promise, output the lattice vector closest to $\boldsymbol{t}$.
		SIS (Short Integer Solution)	Find a short nonzero modular relation	Given $A \in \mathbb{Z}_q^{m \times n}$, find $\boldsymbol{z} \in \mathbb{Z}^n \setminus \{\mathbf{0}\}$ such that $A\boldsymbol{z} \equiv \mathbf{0} \pmod{q}$ and $\|\boldsymbol{z}\|$ is small.
		Search-LWE	Recover the hidden secret from noisy linear equations	Given samples $(\boldsymbol{a}_i, b_i)$ where $b_i = \langle \boldsymbol{a}_i, \boldsymbol{s} \rangle + e_i \pmod{q}$, recover $\boldsymbol{s}$.
	Decision	GapSVP	Decide whether the lattice contains a very short vector	Given (B, d), distinguish between $\lambda_1(\mathcal{L}(B)) \leq d$ and $\lambda_1(\mathcal{L}(B)) > \gamma d$, under a promise.
		GapCVP	Decide whether the target is very close or definitely far	Given $(B, \boldsymbol{t}, d)$, distinguish between $\mathrm{dist}(\boldsymbol{t}, \mathcal{L}(B)) \leq d$ and $> \gamma d$, under a promise.
		Decision-SIS style variant	Decide whether a short modular dependency exists	Given (A, β), decide whether there exists nonzero $\boldsymbol{z}$ with $A\boldsymbol{z} \equiv \mathbf{0} \pmod{q}$ and $\|\boldsymbol{z}\| \leq \beta$.
	Distinguishing	Decision-LWE	Tell LWE samples from uniform random samples	Distinguish $(\boldsymbol{a}, \langle \boldsymbol{a}, \boldsymbol{s} \rangle + e \bmod q)$ from $(\boldsymbol{a}, u)$ where $u \leftarrow \mathbb{Z}_q$ uniformly.
		Decision-RLWE / Decision-MLWE	Distinguish structured noisy algebraic samples from random ones	Distinguish valid RLWE/MLWE samples from uniformly random samples over the same space.
Code-based	Search	Syndrome Decoding (SD)	Find a low-weight error matching the syndrome	Given $H, \boldsymbol{s}, w$, find $\boldsymbol{e}$ such that $H\boldsymbol{e}^T = \boldsymbol{s}^T$ and $w_H(\boldsymbol{e}) \leq w$.
		Nearest Codeword Problem	Find the codeword closest to a received word	Given a code C and received word $\boldsymbol{y}$, output $\boldsymbol{c} \in C$ minimizing Hamming distance $d_H(\boldsymbol{y}, \boldsymbol{c})$.
		Low-Weight Codeword Problem	Find a nonzero codeword of small Hamming weight	Given a linear code C, output nonzero $\boldsymbol{c} \in C$ with small $w_H(\boldsymbol{c})$.
	Decision	Decision-SD	Decide whether a low-weight error exists	Given $(H, \boldsymbol{s}, w)$, decide whether there exists $\boldsymbol{e}$ such that $H\boldsymbol{e}^T = \boldsymbol{s}^T$ and $w_H(\boldsymbol{e}) \leq w$.
		Gap decoding variant	Decide whether decoding radius is small or definitely larger	Distinguish between existence of a solution of weight $\leq w$ and nonexistence of any solution of weight $\leq \gamma w$.
	Distinguishing	Structured-code indistinguishability	Tell whether a public code hides algebraic structure or is random	Distinguish a disguised structured code family from a uniformly random linear code.